

Arquitetura — ByteBank Mobile

Base arquitetural definitiva do app mobile (React Native + Expo), versão mobile do ByteBank (referência web: Next.js + Tailwind + shadcn/ui + Firebase Auth). Este documento é a fonte da verdade sobre como o projeto é organizado e por quê.

Índice

1. [Visão geral](#)
2. [Estrutura completa de pastas](#)
3. [Responsabilidade de cada diretório](#)
4. [Fluxo de autenticação](#)
5. [Fluxo de navegação](#)
6. [Fluxo de dados](#)
7. [Organização dos módulos \(feature-based\)](#)
8. [Organização dos componentes](#)
9. [Organização do Firebase](#)
10. [Convenções de nomenclatura](#)
11. [Como criar uma nova funcionalidade](#)
12. [Boas práticas](#)
13. [O que deve e o que não deve ficar em cada pasta](#)
14. [Exemplos práticos](#)
15. [Stack e decisões de bibliotecas](#)

1. Visão geral

O app é organizado como **arquitetura baseada em features** (feature-based / modular), com o **Expo Router** cuidando apenas de rotear para telas — a lógica de cada funcionalidade vive em `src/modules/<feature>`, não em `src/app/`.

Princípios que guiam todas as decisões abaixo:

- **`src/app/` é só roteamento.** Cada arquivo em `src/app/` é uma casca fina que importa a tela de um module e a renderiza. Nenhuma regra de negócio, chamada a serviço ou estado complexo mora ali.
- **Módulos não se acoplam entre si.** Um module só expõe o que está no seu `index.tsx`. Nenhum outro module (ou tela) pode importar `modules/transfers/components/TransferCard` diretamente — só o que `modules/transfers/index.tsx` exportar publicamente.
- **Telas nunca falam com o Firebase diretamente.** Toda comunicação passa por `module/services/*.ts`, que por sua vez usa os wrappers de `src/firebase/`.
- **Estado de servidor ≠ estado de cliente.** Dados vindos do Firestore vivem em React Query. Context API é reservado para estado de UI/sessão compartilhado entre partes não relacionadas da árvore (autenticação, tema, conectividade). Ver [seção 6](#).
- **Estilização via NativeWind** (Tailwind para React Native) — decisão detalhada na [seção 15](#).

Essa organização é a mesma filosofia do projeto web de referência (separação app-router/components/hooks/services/store/types), adaptada para as particularidades do Expo Router (grupos de rota, tabs, modais) e do React Native (sem DOM, storage assíncrono, gestos nativos).

2. Estrutura completa de pastas

```

tech-challenge/
├─ app.json
├─ tsconfig.json
├─ package.json
├─ tailwind.config.js
├─ docs/
│   └─ architecture.md
├─ assets/                                # mantido na raiz – ver nota abaixo
│   └─ images/
│       └─ expo.icon
└─ src/
    ├─ app/                                # Expo Router – só roteamento
    │   └─ (auth)/                        # stack pública (não autenticada)
    │       └─ _layout.tsx
    │       └─ login.tsx
    │       └─ register.tsx
    │       └─ forgot-password.tsx
    │   └─ (tabs)/                        # bottom tabs autenticadas
    │       └─ _layout.tsx                # <Tabs> com os 4 ícones
    │       └─ index.tsx                  # Home
    │       └─ accounts/
    │           └─ index.tsx
    │           └─ [id].tsx                # stack aninhada dentro da tab
    │       └─ transfers/
    │           └─ index.tsx
    │       └─ profile/
    │           └─ index.tsx
    │   └─ (modals)/                      # modais globais, fora das tabs
    │       └─ _layout.tsx                # Stack com presentation: "modal"
    │       └─ new-transfer.tsx
    │       └─ edit-avatar.tsx
    │   └─ _layout.tsx                    # providers + auth guard (root)
    │       └─ +not-found.tsx
    │
    └─ modules/                          # features – coração da aplicação
        └─ auth/
        └─ home/
        └─ accounts/
        └─ transfers/
        └─ profile/
            └─ components/
            └─ hooks/
            └─ services/
            └─ types.ts
            └─ constants.ts
            └─ validations.ts
            └─ styles.ts
            └─ index.tsx
    └─ components/                      # design system / UI reutilizável
        └─ ui/                          # primitivos: Button, Input, Card, Badge...

```

```

├── feedback/                                # Loading, EmptyState, Skeleton, Toast
│   ├── layout/                             # ScreenContainer, Header, Divider
│   └── navigation/                         # BottomTabItem, TabBar
├── layouts/                                # composições de tela reutilizáveis
│   ├── auth-layout.tsx
│   └── tab-screen-layout.tsx
├── contexts/
│   ├── auth-context.tsx
│   ├── theme-context.tsx
│   ├── connectivity-context.tsx
│   ├── notification-context.tsx
│   └── app-context.tsx
├── hooks/                                  # hooks cross-cutting (não pertencem a 1 module)
│   ├── use-debounce.ts
│   ├── use-network-status.ts
│   └── use-app-state.ts
├── services/                              # camada de serviços cross-cutting
│   ├── query-client.ts
│   └── storage.service.ts
├── firebase/                              # único ponto de contato com o Firebase SDK
│   ├── config.ts
│   ├── auth.ts
│   ├── firestore.ts
│   └── storage.ts
├── lib/                                    # adapters/config de libs de terceiros
│   ├── react-query.ts
│   └── charts.ts
├── utils/                                  # funções puras, sem efeito colateral
│   ├── format-currency.ts
│   ├── format-date.ts
│   └── mask.ts
├── constants/
│   ├── theme.ts                            # já existe no projeto
│   ├── routes.ts
│   └── api.ts
├── types/                                  # tipos globais / compartilhados entre módulos
│   ├── api.ts
│   ├── navigation.ts
│   └── user.ts
├── config/
│   ├── env.ts                             # leitura + validação (zod) das env vars
│   └── app-config.ts

```

```

|
├── validations/                                # schemas Zod compartilhados
│   └── common.schema.ts                       # cpf, telefone, moeda...
|
├── theme/                                     # design tokens – fonte da verdade
│   ├── colors.ts
│   ├── typography.ts
│   └── spacing.ts
|
└── global.css                                # já existe – entrada do NativeWind/Tailwind

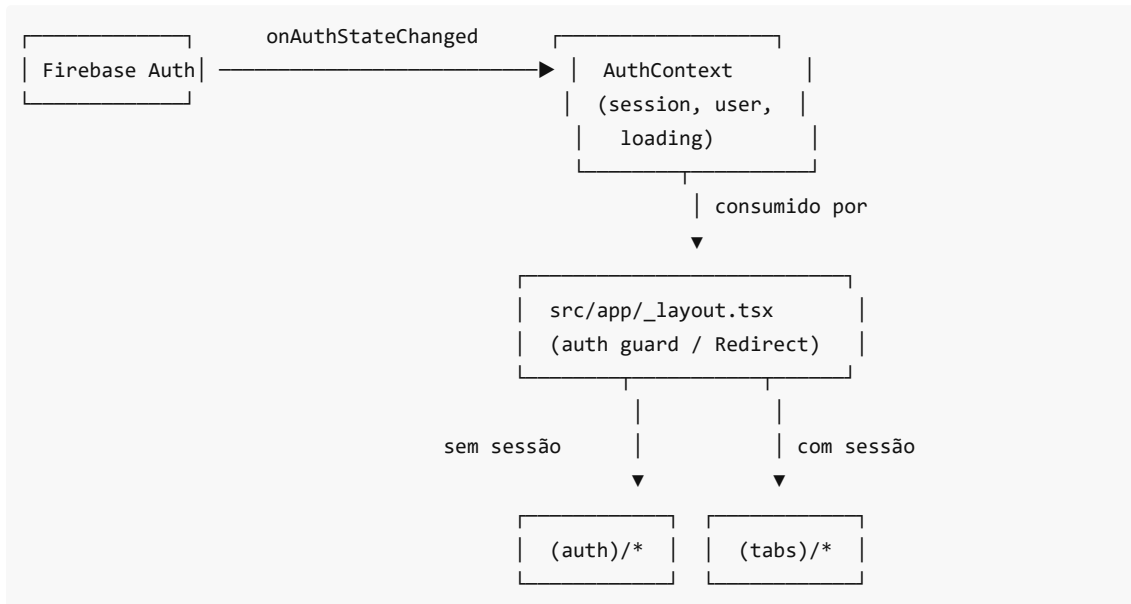
```

Nota sobre `assets/` : a lista original do usuário sugeria `src/assets/` . Mantivemos `assets/` na raiz do projeto porque é a convenção do próprio Expo — `app.json` já referencia `./assets/images/...` e `./assets/expo.icon` , e o template gerado pelo `create-expo-app` espera esse caminho. Mover para dentro de `src/` exigiria reconfigurar `app.json` sem nenhum ganho real; é um desvio deliberado da proposta inicial.

3. Responsabilidade de cada diretório

Pasta	Responsabilidade
<code>src/app</code>	Rotas do Expo Router. Define onde uma tela aparece na navegação, não o que ela faz.
<code>src/modules</code>	Uma pasta por funcionalidade de negócio. Tudo que uma feature precisa (UI, hooks, serviços, tipos, validação) vive junto.
<code>src/components</code>	Componentes de UI sem conhecimento de domínio — reusáveis em qualquer module.
<code>src/layouts</code>	Composições estruturais de tela (ex.: header + safe area + tab bar) reaproveitadas por várias rotas.
<code>src/contexts</code>	Estado de cliente compartilhado por partes não relacionadas da árvore (sessão, tema, conectividade).
<code>src/hooks</code>	Hooks genéricos, sem relação com uma feature específica.
<code>src/services</code>	Serviços que não pertencem a nenhum module isolado (cliente do React Query, wrapper de storage).
<code>src/firebase</code>	Toda a integração com Firebase. Nenhum outro arquivo do projeto importa o SDK do Firebase diretamente.
<code>src/lib</code>	Configuração/adaptação de bibliotecas de terceiros (instâncias, plugins).
<code>src/utlis</code>	Funções puras e determinísticas, sem I/O, sem hooks, sem estado.
<code>src/constants</code>	Valores fixos usados em várias partes do app (rotas nomeadas, endpoints, tema base).
<code>src/types</code>	Contratos TypeScript compartilhados entre módulos . Tipos usados por um único module ficam no <code>types.ts</code> do próprio module.
<code>src/config</code>	Configuração de ambiente/app (env vars tipadas, flags de build).
<code>src/validations</code>	Schemas Zod reaproveitados por mais de um module (ex.: validação de CPF usada em auth e profile).

4. Fluxo de autenticação



1. `src/firebase/auth.ts` expõe funções finas (`signIn`, `signUp`, `signOut`, `subscribeToAuthChanges`) sobre o SDK do Firebase Auth.
2. `modules/auth/services/auth.service.ts` usa essas funções e traduz erros do Firebase em mensagens de domínio (ex.: `auth/wrong-password` → "Credenciais inválidas").
3. `AuthContext` (`src/contexts/auth-context.tsx`) assina `subscribeToAuthChanges` uma vez, no topo da árvore, e expõe `{ user, status: 'loading' | 'authenticated' | 'unauthenticated', signIn, signOut }`.
4. `src/app/_layout.tsx` lê `AuthContext` e decide qual grupo de rota é acessível — enquanto `status === 'loading'`, mostra o splash; depois disso, redireciona para `(auth)` ou `(tabs)`.
5. Token de sessão sensível é persistido com `expo-secure-store` (não `AsyncStorage`) — ver [seção 15](#).
6. Formulários de login/cadastro ficam em `modules/auth/components`, validados com `react-hook-form` + `zod` (`modules/auth/validations.ts`).

5. Fluxo de navegação

- **Grupo (auth)** — stack pública: `login`, `register`, `forgot-password`. Sem tab bar.
- **Grupo (tabs)** — `<Tabs>` do Expo Router com 4 itens fixos: **Home**, **Contas**, **Transferências**, **Perfil** (o "menu inferior" mencionado no domínio do app — bottom tab navigation, não uma sidebar lateral). Cada tab pode ter sua própria stack aninhada (ex.: `accounts/index.tsx` → `accounts/[id].tsx`).
- **Grupo (modals)** — telas apresentadas como modal (`Stack.Screen options={{ presentation: 'modal' }}`) para fluxos curtos e interruptivos: nova transferência, confirmação de transação, edição de avatar. Ficam fora de `(tabs)` para não herdar a tab bar.
- **_layout.tsx raiz** — monta os providers globais (Auth, React Query, Theme, Connectivity) e faz o `auth guard` entre `(auth)` e `(tabs)`.
- **+not-found.tsx** — já existe no template, cobre rotas inexistentes.

Regra prática: um arquivo em `src/app/**` nunca deve ultrapassar ~15-20 linhas. Se crescer mais que isso, a lógica pertence ao module.

6. Fluxo de dados

```
Screen (src/app)
  | importa
  ▼
Module screen (modules/<feature>/index.tsx)
  | usa
  ▼
Module hook (modules/<feature>/hooks/use-accounts.ts) → React Query (cache/estado
servidor)
  | chama
  ▼
Module service (modules/<feature>/services/accounts.service.ts)
  | chama
  ▼
src/firebase/firestore.ts (wrapper fino do SDK)
  |
  ▼
Firebase (Firestore / Auth / Storage)
```

- **Estado de servidor** (dados que vêm do Firestore/Auth/Storage): sempre via `@tanstack/react-query`, dentro de hooks do próprio module (`use-accounts.ts`, `use-transfers.ts`). Nunca duplicado em Context ou `useState` manual.
 - **Estado de cliente/UI compartilhado** entre módulos: Context API (sessão, tema, conectividade, notificações).
 - **Estado local de tela**: `useState` / `useReducer` dentro do próprio componente — não sobe para Context nem para um module hook se só um componente usa.
 - **Formulários**: `react-hook-form` controla o estado do form; `zod` valida; o `onSubmit` chama um module hook, que chama o service.
-

7. Organização dos módulos (feature-based)

Cada pasta em `src/modules/<feature>` segue o mesmo template:

```
modules/transfers/
├── components/      # UI específica desta feature (TransferForm, TransferListItem)
├── hooks/           # use-transfers.ts, use-create-transfer.ts (React Query)
├── services/        # transfers.service.ts (chama src/firebase)
├── types.ts         # Transfer, TransferStatus, CreateTransferInput
├── constants.ts     # limites, enums de UI, chaves de query
├── validations.ts   # createTransferSchema (zod)
├── styles.ts        # classes NativeWind reaproveitadas dentro do module (opcional)
└── index.tsx        # API pública do module – o que outros arquivos podem importar
```

Quando usar cada subpasta:

- `components/` — só entra aqui um componente que não faz sentido fora deste module (ex.: `TransferConfirmationCard`). Se um componente puder ser genérico, ele pertence a `src/components`, não ao module.
- `hooks/` — hooks de dados (React Query) e hooks de UI específicos da feature.
- `services/` — a única camada autorizada a chamar `src/firebase/*`. Retorna dados já tipados pelo `types.ts` do module, nunca o formato cru do Firestore.
- `types.ts` — tipos usados **só** dentro deste module. Se `accounts` e `transfers` precisarem do mesmo tipo, ele sobe para `src/types`.
- `constants.ts` — valores fixos da feature (ex.: `MAX_TRANSFER_AMOUNT`, chaves de cache do React Query).
- `validations.ts` — schemas Zod específicos da feature. Validações genéricas (CPF, telefone) vêm de `src/validations/common.schema.ts` e são compostas aqui.
- `index.tsx` — barrel/API pública. Exporta a tela principal (usada por `src/app`) e qualquer componente/hook que outro module tenha real necessidade de reaproveitar. **Tudo que não é exportado por aqui é privado do module.**

8. Organização dos componentes

`src/components` é dividido por categoria, não por feature (isso é o que o diferencia de `src/modules/*/components`):

- `ui/` — primitivos de design system: `Button`, `Input`, `Card`, `Modal`, `Avatar`, `Badge`, `Typography`, `Divider`. Sem conhecimento de domínio bancário.
- `feedback/` — estados de tela: `Loading`, `Skeleton`, `EmptyState`, `Toast` / `Snackbar`.
- `layout/` — estrutura de tela: `ScreenContainer` (safe area + padding padrão), `Header`.
- `navigation/` — peças de navegação reaproveitáveis: `BottomTabItem`, `TabBar` custom.

Regra anti-duplicação: antes de criar um componente dentro de um module, verificar se ele já existe (ou deveria existir) em `src/components`. Um componente só nasce dentro de um module se for genuinamente específico daquela feature (ex.: `AccountBalanceChart` não faz sentido em `src/components/ui`, mas um `Card` genérico sim). Se, depois de criado dentro de um module, o mesmo componente passa a ser necessário em outra feature, ele é **promovido** para `src/components` — nunca duplicado.

9. Organização do Firebase

```
src/firebase/
├─ config.ts      # initializeApp – única inicialização do SDK no projeto
├─ auth.ts        # signIn, signUp, signOut, subscribeToAuthChanges, getCurrentUser
├─ firestore.ts   # helpers genéricos: getDocument, setDocument, queryCollection,
  subscribeToDocument
└─ storage.ts     # uploadFile, getDownloadUrl, deleteFile
```

Regras:

- **Nenhuma tela ou componente importa `firebase/*` (SDK) ou `src/firebase/*` diretamente.** Só `modules/<feature>/services/*.ts` pode importar `src/firebase/*`.
- `src/firebase/firestore.ts` expõe funções genéricas por **operação**, não por domínio (ex.: `queryCollection<T>(path, constraints)`), para não vazar conhecimento de "conta" ou "transferência" nesta camada.

- Regras de negócio (ex.: "uma transferência não pode ser maior que o saldo") vivem no `modules/transfers/services/transfers.service.ts`, nunca em `src/firebase`.
- Cloud Functions futuras entram como `src/firebase/functions.ts`, seguindo o mesmo padrão de wrapper fino + chamada pelos services dos módulos.
- Regras de segurança do Firestore/Storage (`firestore.rules`, `storage.rules`) ficam na raiz do projeto, fora de `src/` — não fazem parte do bundle do app.

10. Convenções de nomenclatura

Item	Convenção	Exemplo
Arquivos	kebab-case (já é o padrão do projeto)	<code>themed-text.tsx</code> , <code>use-accounts.ts</code>
Componentes (export)	PascalCase	<code>export function TransferForm()</code>
Hooks	camelCase prefixado com use	<code>useAccounts</code> , <code>useCreateTransfer</code>
Contexts	PascalCase + sufixo Context/Provider	<code>AuthContext</code> , <code>AuthProvider</code>
Rotas dinâmicas (Expo Router)	<code>[param].tsx</code>	<code>accounts/[id].tsx</code>
Grupos de rota	(nome)	<code>(auth)</code> , <code>(tabs)</code> , <code>(modals)</code>
Tipos/Interfaces	PascalCase, sem prefixo I	<code>Transfer</code> , <code>CreateTransferInput</code>
Schemas Zod	camelCase + sufixo Schema	<code>createTransferSchema</code>
Constantes	SCREAMING_SNAKE_CASE para valores fixos; camelCase para objetos de config	<code>MAX_TRANSFER_AMOUNT</code> , <code>queryKeys</code>
Barrel de module	sempre <code>index.tsx</code> (ou <code>index.ts</code> se não renderiza JSX)	<code>modules/transfers/index.tsx</code>

11. Como criar uma nova funcionalidade

Exemplo: adicionar a feature "Investimentos".

1. Criar `src/modules/investments/` com `components/`, `hooks/`, `services/`, `types.ts`, `constants.ts`, `validations.ts`, `index.tsx`.
2. Definir os tipos em `types.ts` (`Investment`, `InvestmentType` ...).
3. Implementar `services/investments.service.ts`, chamando `src/firebase/firestore.ts`.
4. Implementar `hooks/use-investments.ts` com `useQuery` / `useMutation` do React Query, consumindo o service.
5. Construir a UI em `components/` reaproveitando `src/components/ui/*`; montar a tela principal e exportá-la em `index.tsx`.
6. Criar a rota em `src/app/(tabs)/investments/index.tsx` (ou como stack aninhada), importando a tela do module — nada além de um import + render.
7. Se a feature aparece na tab bar, adicionar o item em `src/app/(tabs)/_layout.tsx` e em `src/components/navigation/BottomTabItem` se precisar de um ícone novo.

8. Se a feature reaproveita validação comum (ex.: valores monetários), importar de `src/validations/common.schema.ts` em vez de recriar.

12. Boas práticas

- Um module nunca importa arquivos internos de outro module — só o `index.tsx` dele.
- Toda tela (`src/app/**`) é "burra": só compõe layout + importa a tela do module.
- Toda chamada ao Firebase passa por um `service` ; nenhum hook ou componente chama o SDK direto.
- Context API só para estado verdadeiramente global e de baixa frequência de mudança — Context para estado que muda a cada tecla digitada gera re-renders desnecessários em toda a árvore.
- Dados de servidor sempre em React Query — nunca copiados para `useState` "para guardar".
- Toda validação de formulário usa `zod` + `react-hook-form` , nunca validação manual em `onChangeText` .
- Tokens/sessão sensíveis vão para `expo-secure-store` , nunca `AsyncStorage` .
- Estilos usam classes `NativeWind`; `StyleSheet.create` só para casos que o Tailwind não cobre bem (animações via `Reanimated`, valores calculados em runtime).
- Nenhum `console.log` ou dado sensível (token, saldo, CPF) em logs de produção.
- Cada PR que adiciona uma feature nova deve vir com seu module completo (não fatiar um module entre vários PRs incompletos).

13. O que deve e o que não deve ficar em cada pasta

Pasta	✅ Deve conter	❌ Não deve conter
<code>src/app</code>	Rotas finas, layouts de navegação, guards	Regras de negócio, chamadas a serviço, estado complexo
<code>src/modules/*</code>	Tudo específico da feature	Componentes/hooks genéricos reaproveitáveis por outras features
<code>src/components</code>	UI genérica, sem conhecimento de domínio	Componentes que sabem o que é "conta" ou "transferência"
<code>src/contexts</code>	Estado global de baixa frequência	Estado de servidor (isso é do React Query) ou estado de uma tela só
<code>src/firebase</code>	Wrappers finos do SDK, por operação	Regras de negócio, formatação de dados para UI
<code>src/utils</code>	Funções puras	Funções que fazem fetch, leem Context ou têm efeito colateral
<code>src/types</code>	Tipos usados por 2+ módulos	Tipos usados por um único module (ficam no <code>types.ts</code> dele)
<code>src/validations</code>	Schemas Zod reaproveitados	Schema específico de uma única tela/module
<code>src/constants</code>	Valores fixos cross-cutting	Valores específicos de uma feature (ficam no <code>constants.ts</code> do module)

14. Exemplos práticos

14.1 Rota fina (src/app/(tabs)/transfers/index.tsx)

```
import { TransfersScreen } from '@modules/transfers';

export default function Transfers() {
  return <TransfersScreen />;
}
```

14.2 Module hook consumindo o service via React Query

```
// src/modules/transfers/hooks/use-transfers.ts
import { useQuery } from '@tanstack/react-query';
import { getTransfers } from '../services/transfers.service';
import { transfersKeys } from '../constants';

export function useTransfers(accountId: string) {
  return useQuery({
    queryKey: transfersKeys.list(accountId),
    queryFn: () => getTransfers(accountId),
  });
}
```

14.3 Service falando com o Firebase

```
// src/modules/transfers/services/transfers.service.ts
import { queryCollection } from '@firebase/firestore';
import type { Transfer } from '../types';

export function getTransfers(accountId: string) {
  return queryCollection<Transfer>('transfers', [
    { field: 'accountId', op: '==', value: accountId },
  ]);
}
```

14.4 Barrel público do module

```
// src/modules/transfers/index.tsx
export { TransfersScreen } from './components/transfers-screen';
export { useTransfers } from './hooks/use-transfers';
export type { Transfer } from './types';
```

14.5 Contexto de autenticação (uso)

```
// src/app/_layout.tsx (trecho)
const { status } = useAuthContext();
```

```

if (status === 'loading') return <SplashScreen />;

return (
  <Stack>
    <Stack.Protected guard={status === 'authenticated'}>
      <Stack.Screen name="(tabs)" />
    </Stack.Protected>
    <Stack.Protected guard={status === 'unauthenticated'}>
      <Stack.Screen name="(auth)" />
    </Stack.Protected>
  </Stack>
);

```

15. Stack e decisões de bibliotecas

15.1 Estilização — recomendação oficial: NativeWind

Opção	Prós	Contras
NativeWind (recomendado)	Compila para <code>StyleSheet</code> nativo (boa performance); sintaxe Tailwind já usada no projeto web de referência (curva de aprendizado ~zero pra quem já viu o ByteBank web); <code>src/global.css</code> já existe no projeto, sinalizando que era o plano original; integra bem com Expo Router, Reanimated e Gesture Handler; comunidade grande, muito conteúdo de apoio	Depende de plugin Babel/Metro; alguns componentes de terceiros exigem <code>cssInterop</code> para aceitar <code>className</code>
Styled Components / Emotion	API familiar para quem vem do CSS-in-JS web	CSS-in-JS roda em runtime no RN — pior performance; sem extração estática; hoje é considerado abordagem legada para RN
Tamagui	Compilador próprio, ótima performance, já vem com design system e animações	Curva de aprendizado maior, setup mais pesado, overkill para o escopo atual do projeto
Unistyles v3	Baseado em compilador/JSI, muito rápido, zero re-render em troca de tema	Ecossistema e comunidade menores — mais difícil achar solução pronta para um time estudantil
<code>StyleSheet</code> nativo puro	Zero dependências, performance-baseline	Verboso, sem tokens/utilitários — difícil manter consistência visual em equipe

Decisão: NativeWind v4. Justificativa consolidada: o projeto já tem `global.css` esperando por ele, o time ganha continuidade de linguagem com o front-end web (que usa Tailwind), e o resultado final compila para `StyleSheet`,

então não há penalidade de performance em runtime. `tailwind.config.js` na raiz do projeto lê os tokens de `src/theme/*` como fonte única da verdade (cores, espaçamento, tipografia).

15.2 Bibliotecas adicionais recomendadas

Biblioteca	Motivo
<code>@tanstack/react-query</code>	Separar estado de servidor (Firestore) do Context API — cache, refetch, retry e loading/error state prontos, sem reinventar isso à mão.
<code>expo-secure-store</code>	Um app bancário não deve guardar token de sessão em <code>AsyncStorage</code> (não criptografado). <code>AsyncStorage</code> fica reservado a preferências não sensíveis (ex.: tema escolhido).
<code>react-native-gifted-charts</code>	Gráficos do dashboard financeiro (evolução de saldo, categorização de gastos) sem exigir Skia — mais simples de manter que <code>victory-native-x1</code> para o escopo atual.
<code>date-fns</code>	Manipulação e formatação de datas no extrato/filtros, com tree-shaking melhor que <code>moment</code> .
<code>@hookform/resolvers</code>	Conecta zod ao <code>react-hook-form</code> (<code>zodResolver</code>), já previstos no stack.
<code>eslint-plugin-boundaries</code> (ou regra de import equivalente)	Garante em CI que nenhum module importa arquivo interno de outro module — a regra "só via <code>index.tsx</code> " vira lint, não confiança.

Considerar mais adiante, não instalar agora: `zustand` (só se o Context de sessão/tema começar a causar re-renders perceptíveis em telas de lista grandes), `react-native-mmkv` (troca de performance pelo `AsyncStorage` se o app crescer), Sentry/Bugsnag (observabilidade em produção), `jest` + `@testing-library/react-native` (testes automatizados).

15.3 Context API — quais contexts existem e quando usar Context vs estado local

Context	Para quê
<code>AuthContext</code>	Sessão do usuário, status de autenticação, ações de login/logout.
<code>ThemeContext</code>	Fina camada sobre o color scheme (claro/escuro), sincronizada com as classes <code>dark</code> : do <code>NativeWind</code> .
<code>ConnectivityContext</code>	Estado online/offline (via <code>@react-native-community/netinfo</code>), para banners e bloqueio de ações que exigem rede.
<code>NotificationContext</code>	Permissões e fila de notificações in-app/push.
<code>AppContext</code>	Flags globais de app (onboarding já visto, feature flags, força de atualização).

Critério: Context API é para estado lido por partes não relacionadas da árvore de componentes e que muda com pouca frequência. Estado de servidor (o que vem do Firestore) nunca vai para Context — isso é responsabilidade do React Query. Estado local (`useState` / `useReducer`) é o padrão para qualquer coisa que só uma tela ou um componente usa; só sobe para Context quando dois ou mais lugares desconectados da árvore precisam do mesmo dado.